HIGH-LEVEL DESIGN

RAG-Based Customer Support Assistant

with Conditional Routing & HITL Escalation

Version 1.0 | LangGraph + ChromaDB + Phi-2

1. System Overview

1.1 Problem Definition

Customer support operations face a persistent challenge: handling high volumes of document-centric queries accurately and consistently. This system addresses that by automating responses to customer queries against a proprietary PDF knowledge base, reducing human agent load while ensuring answer fidelity through grounding checks and escalation when confidence is insufficient.

1.2 System Scope

IN SCOPE:

- PDF document ingestion and chunking into a searchable vector store
- Semantic retrieval with cross-encoder re-ranking
- Intent detection across 5 categories: complaint, summary, process, faq, unknown
- LLM answer generation using Microsoft Phi-2 (2.7B parameters)
- Semantic grounding verification of generated answers
- Confidence-based conditional routing and HITL escalation
- CLI-based interactive query loop

OUT OF SCOPE:

- Web/API interface (not present in implementation)
- Multi-document simultaneous support (single PDF per session)
- Persistent conversation memory across sessions
- Formal LangGraph StateGraph construction (workflow is implemented as sequential function calls)
- Human agent response integration (escalation only emits a tagged string)

2. Architecture Diagram

The following diagram represents the end-to-end system architecture as implemented in the notebook:

```
┌──────────────────────────────────────────────────────────────────────┐
│                    USER (CLI Input Loop)                             │
└──────────────────────────────────────────────────────────────────────┘
                          │ raw query string
                          ▼
┌──────────────────────────────────────────────────────────────────────┐
│              DOCUMENT INGESTION PIPELINE                             │
│  PyPDFLoader → RecursiveCharacterTextSplitter (500/50)               │
└──────────────────────────────────────────────────────────────────────┘
                          │ chunks[]
                          ▼
┌──────────────────────────────────────────────────────────────────────┐
│           EMBEDDING + VECTOR STORE LAYER                             │
│    HuggingFaceEmbeddings (all-MiniLM-L6-v2) → ChromaDB               │
└──────────────────────────────────────────────────────────────────────┘
                          │
     ┌────────────────────────────────────────────────────────────┐
     │           QUERY PROCESSING PIPELINE                        │
     │                                                            │
  rewrite_query()           detect_intent()                       │
  (Phi-2 LLM)           (keyword-based classifier)                │
     │                        │                                   │
     └────────────────────────┘                                   │
                  │ retrieve(query, k=5)                          │
                  │ → similarity_search_with_score                │
                  │ → CrossEncoder rerank (top-3)                 │
                  ▼                                               │
┌──────────────────────────────────────────┐                      │
│        CONDITIONAL ROUTER                │                      │
│  intent == complaint → [ESCALATED]       │                      │
│  docs empty → [ESCALATED]                │                      │
│  confidence < 0.4 → [ESCALATED]          │                      │
│  not grounded → soft fallback            │                      │
│  unknown intent → clarification request  │                      │
│  else → LLM Answer Generation            │                      │
└──────────────────────────────────────────┘                      │
                  │                                               │
     ┌────────────┴────────────┐                                  │
     ▼                         ▼                                  │
  [ANSWER]                 [ESCALATED]  ←──────────────────────────┘
+ DEBUG INFO                HITL Tag
+ SOURCES
```

3. Component Descriptions

3.1 Document Loader

Attribute	Detail
Implementation	langchain.document_loaders.PyPDFLoader
Input	Single PDF file uploaded via Google Colab files.upload()
Output	List of LangChain Document objects (one per PDF page)
Interaction	Feeds raw document list to the Chunking module
Limitation	Single-file per session; no batch ingestion

3.2 Chunking Module

Attribute	Detail
Implementation	langchain.text_splitter.RecursiveCharacterTextSplitter
Chunk Size	500 characters
Chunk Overlap	50 characters
Output	List of smaller Document chunks with preserved metadata
Strategy	Recursive splitting: paragraph → sentence → word boundaries

3.3 Embedding Model

Attribute	Detail
Model	sentence-transformers/all-MiniLM-L6-v2
Provider	HuggingFaceEmbeddings (LangChain Community)
Output Dimension	384-dimensional dense vector
Usage	Encodes both document chunks (at index time) and queries (at retrieval time)
Also used in	semantic_grounding() — cosine similarity between answer and context

3.4 Vector Store

Attribute	Detail
Implementation	langchain_community.vectorstores.Chroma (ChromaDB v0.4.22)
Storage	In-memory (no persistence configured)
Index type	Default HNSW via ChromaDB's embedded DuckDB+Parquet

Attribute	Detail
Operations	similarity_search_with_score(query, k=5) — returns top-5 chunks + L2 scores
Initialization	Chroma.from_documents(chunks, embedding_model)

3.5 Retriever & Re-ranker

Attribute	Detail
First-stage	ChromaDB similarity_search_with_score → top-5 candidates
Re-ranker model	cross-encoder/ms-marco-MiniLM-L-6-v2 (CrossEncoder)
Re-ranker output	Top-3 passages by relevance score (pointwise scoring)
Confidence metric	Mean of L2 scores from first-stage retrieval
Threshold	confidence < 0.4 triggers HITL escalation

3.6 LLM

Attribute	Detail
Model	microsoft/phi-2 (2.7B parameters)
Loading	AutoModelForCausalLM with device_map=auto, torch_dtype=float16
Tokenizer	AutoTokenizer from transformers (max_length=1024)
Generation params	max_new_tokens=150, temperature=0.3, top_p=0.9, do_sample=True
Used for	rewrite_query() and generate_answer()

3.7 Workflow Engine

NOTE: Despite LangGraph being listed as a dependency (langgraph==0.0.32), no formal StateGraph, nodes, or edges are constructed in the notebook. The workflow is implemented as a sequential Python function chat() that orchestrates all components imperatively.

3.8 Routing Logic

Condition	Action
docs empty OR context < 50 chars	[ESCALATED] No relevant information found
intent == 'complaint'	[ESCALATED] Complaint detected → Human support required
semantic_grounding returns False	Soft fallback message (not tagged as escalation)
confidence < 0.4	[ESCALATED] Low confidence retrieval

Condition	Action
intent == 'unknown'	Clarification request returned to user
All checks pass	Answer + DEBUG INFO + SOURCES returned

3.9 HITL Module

HITL (Human-in-the-Loop) is implemented as a string-tagging mechanism. When escalation conditions are met, the chat() function returns a string prefixed with [ESCALATED]. There is no actual human agent queue, webhook, or callback mechanism in the implementation. The HITL tag signals to an external system (not implemented) that a human should handle this query.

4. End-to-End Data Flow

4.1 Ingestion Pipeline: PDF → Vector Store

Step	Function/Component	Input	Output
1	files.upload() [Colab]	User uploads PDF	file_name string
2	PyPDFLoader.load()	PDF file path	List[Document] (per page)
3	RecursiveCharacterTextSplitter.split_documents()	List[Document]	List[Chunk] (500-char segments)
4	HuggingFaceEmbeddings.embed_documents()	List[Chunk] texts	List[384-d float vectors]
5	Chroma.from_documents()	Chunks + vectors	In-memory ChromaDB index

4.2 Query Lifecycle: User Query → Response

Step	Function	Action
1	rewrite_query()	Phi-2 rewrites raw query to a precise document-search question
2	retrieve(query, k=5)	ChromaDB returns top-5 chunks + L2 similarity scores
3	rerank(query, docs, top_k=3)	CrossEncoder scores pairs, returns top-3 passages
4	detect_intent(query)	Keyword matching classifies into 5 intent categories
5	compute_confidence(scores)	Mean of L2 scores from retrieval step
6	Context emptiness check	If context < 50 chars → [ESCALATED]
7	build_prompt() + generate_answer()	Phi-2 generates answer constrained to context
8	semantic_grounding()	Cosine similarity between answer and context embeddings
9	Conditional routing	Route to answer, soft fallback, or [ESCALATED] per rules
10	Response assembly	Answer + DEBUG INFO (intent, confidence, grounded) + SOURCES

5. Technology Choices — Justified

Technology	Version	Justification
ChromaDB	0.4.22	Embedded, zero-config vector DB suitable for Colab prototyping; no external server needed; native LangChain integration
all-MiniLM-L6-v2	sentence-transformers	Lightweight 384-d model; runs on CPU; excellent semantic quality for English retrieval tasks; open-source
CrossEncoder ms-marco-MiniLM-L-6-v2	sentence-transformers	State-of-the-art re-ranker for MS MARCO passage ranking; significantly improves precision over bi-encoder alone
Microsoft Phi-2	2.7B params	Compact LLM that fits in Colab GPU memory (float16); strong instruction-following for its size; no API cost
LangChain	0.1.16	Provides loaders, splitters, vector store wrappers; reduces boilerplate for PDF-to-Chroma pipeline
LangGraph	0.0.32	Listed as dependency; imported but no formal graph constructed — workflow is procedural via chat()
PyTorch	2.2.1	Backend for both embedding model and Phi-2; float16 support reduces GPU memory footprint

6. Scalability Considerations

6.1 Document Scale

- Current: Single PDF per session, in-memory ChromaDB — no persistence
- Bottleneck: Chroma.from_documents() re-indexes on every session start
- At scale: ChromaDB persistent client + incremental upsert required
- Chunking at 500 chars handles ~200 page documents comfortably in memory

6.2 Query Throughput

- Phi-2 with max_new_tokens=150 on a single T4 GPU: ~2-4 seconds per query
- CrossEncoder re-ranking is the second-largest latency contributor (~0.5-1s for 5 candidates)
- No batching or async processing — fully synchronous
- rewrite_query() doubles LLM calls per turn (one for rewrite, one for answer)

6.3 Latency Breakdown (Estimated)

Stage	Estimated Latency	Notes
ChromaDB similarity search	< 50ms	In-memory HNSW, fast for small corpora
CrossEncoder re-ranking	500ms–1s	Phi-2 model, 5 candidate pairs
Query rewrite (Phi-2)	2–4s	Full LLM inference, major bottleneck
Answer generation (Phi-2)	2–4s	Full LLM inference, major bottleneck
Semantic grounding check	~100ms	Embedding cosine similarity only
Total end-to-end	~5–10s	Dominated by dual Phi-2 calls